

Overview

`packages/graph-core/src/schema.ts` now contains a full validation pipeline for knowledge graphs.

Key additions include:

- `import { z } from "zod"` (R1).
- `EdgeTypeSchema` (R4-14) defines 35 edge types.
- Alias maps for node types, edge types, complexity, and direction (R17-140).
- Sanitization (`sanitizeGraph` , R148-194), normalization (`normalizeGraph` , R467-502), and auto-fix (`autoFixGraph` , R196-354).
- Validation orchestration (`validateGraph` , R504-701) that produces a `ValidationResult` .
- Helper functions `buildInvalidCollectionIssue` and `buildErrors` (R452-465).
- Exported interfaces `GraphIssue` and `ValidationResult` (R436-449).

Key changes

- **Schema definitions** – `EdgeTypeSchema` exposes a 35-value enum (R4-14).
- **Alias maps** – `NODE_TYPE_ALIASES` , `EDGE_TYPE_ALIASES` , `COMPLEXITY_ALIASES` , `DIRECTION_ALIASES` (R17-140) translate common LLM terms to canonical values.
- **Sanitization** – `sanitizeGraph` normalizes nulls, lower-cases enums, and removes optional fields (R148-194).
- **Auto-fix** – `autoFixGraph` supplies defaults for missing fields, coerces types, and records `GraphIssue` objects (R196-354).
- **Normalization** – `normalizeGraph` replaces aliased types in nodes/edges (R467-502).
- **Validation pipeline** – `validateGraph` orchestrates sanitization, normalization, auto-fix, collection checks, project metadata validation, and per-entity validation, returning `ValidationResult` (R504-701).
- **Error handling helpers** – `buildInvalidCollectionIssue` and `buildErrors` centralize fatal and non-fatal issue creation (R452-465).
- **Exported interfaces** – `GraphIssue` and `ValidationResult` provide structured diagnostics (R436-449).

Impact

- **Correctness** – Multi-tier validation catches malformed collections, missing metadata, and invalid references, reducing downstream errors.
- **Observability** – Rich `GraphIssue` logs (levels: `auto-corrected` , `dropped` , `fatal`) aid debugging and telemetry.
- **Maintainability** – Centralized alias maps and schema definitions simplify future updates to node/edge vocabularies.

- **Performance** – Multiple passes (sanitize → normalize → auto-fix) add overhead; data sizes are modest, and early error detection outweighs the cost.

Risks & follow-ups

- **Alias collisions** – Verify that `EDGE_TYPE_ALIASES` and `NODE_TYPE_ALIASES` do not map distinct terms to the same canonical value (deterministic risk signal).
- **Auto-fix side effects** – Defaulting `type` to `"file"` or `weight` to `0.5` may mask genuine data issues; consider a flag to disable auto-fix in production.
- **Schema drift** – External consumers relying on the old graph shape may break; document the new `KnowledgeGraphSchema` contract.
- **Test coverage** – Add unit tests for each validation tier, especially non-array collections and missing project metadata.